

我完成了所有内容.

还有声卡 > ω <

咚咚咚咚咚咚咚咚 咚咚咚咚咚

必答题

YEMU

- 状态机

表示为(r0, r1, My, Mx, Mz) (?,?, 33, 16, 0) → (33,?, 33, 16, 0) → (33, 33, 33, 16, 0) → (16, 33, 33, 16, 0) → (49, 33, 33, 16, 0) → (49, 33, 33, 16, 49)

- 如何执行

取指 → 解析操作码 → 解析操作数 → 执行与写入 → 更新 PC

指令执行过程

已关闭所有 trace

内存读出指令与 snpc 修改 -> 解析指令 -> 解析 imm 与 reg -> 执行与访存与 dnpc 修改 -> 更新 PC

打字小游戏如何运行

- 状态机视角

按下下一个字母按键后状态机的转移将会"分叉", 然后继续状态转移

- 抽象层视角

- 程序: 计时, 读取键盘输入, 比较输入与目标字符, 更新画面进行渲染
- 运行时环境: printf: 在控制台输出提示, rand 生成随机数, ioread: 读取键盘输入, etc., iowrite: 输出到屏幕
- AM: 访问 AM_INPUT_KEYBRD, AM_GPU_FBDRW 等寄存器对应的 MMIO 地址
- ISA: 指令访问内存
- NEMU: 按键通过 SDL 读取 key 并放入 key_queue, 收到读取请求时从 key_queue 中取出放入对应 MMIO 地址; 根据 vmem 进行绘制

编译与链接

- static&inline

- 去掉 static: 没有任何影响???
- 去掉 inline: error: 'inst_fetch' defined but not used, 关掉 Wall 没有任何影响???
- 全去掉: multiple definition of 'inst_fetch'
- 我的解释:
 - 有 inline 的函数会被编译器在调用处展开, 不存在符号表中, 所以不会有重复定义
 - static 函数只在当前文件中可见, 但是链接的时候连了过去, 所以也不会有重复定义, 且能正常调用
- 证明: 查符号表

- volatile
 - $n = (\text{size_after} - \text{size_before}) / \text{sizeof}(\text{int}) = (59120 - 57536) / 4 = 396$
 - $n = (\text{size_after_2} - \text{size_before}) / \text{sizeof}(\text{int}) = (59120 - 57536) / 4 = 396$
 - 原因: common.h 和 debug.h 相互引用, common.h 中的 dummy 被视为 debug.h 中的 dummy, 或者相反 (隐式的加了 extern?)
 - error: redefinition of 'dummy'. 赋初值后两处的 dummy 被视为在两个头中分别定义的变量 (extern 变量不能赋初值)

了解 Makefile

- 设置 NAME = hello
- 设置 SRCS = hello.c
- 包含 \$(AM_HOME)/Makefile 并解析
 - 运行前面的一堆检查
 - 抽取 ISA 和平台
 - 设置 IMAGE
 - 把每个 SRCS 拼接成.o 文件的路径, 保存到 OBJS
 - 把 LIBS 加上 am 和 klilb, 去掉重复元素
 - 设置 LINKAGE 为 OBJS 和 \$(AM_HOME)/每个 LIBS 名字/build/这个 LIBS 名字-架构.a
 - 设置 INC_PATH 为 \$(AM_HOME)/每个 LIBS 名字/include 和当前工作目录下的 include
 - 设置 INCFLAGS 为 -I 每个 INC_PATH, 后面在编译的时候指示去哪找头文件
 - 调用第一条伪命令 image 而调用 image-dep 而编译 \$(LIBS) 和 \$(IMAGE).elf
 - 编译 \$(LIBS) 时以 archive 调用自身脚本, 不包含 hello 和 hello.c
 - 编译 \$(OBJS)
 - 用 gcc 编译每个.c 文件, 生成.o 文件
 - 用 ar 打包成.a 文件
 - 编译 \$(IMAGE).elf, 其依赖 \$INKAGE, 从而编译其中.o 对应的.c
 - 把.o 和 .a 链接成可执行文件

新发现

- gnuc 特供语句表达式
- 字面量默认进行 ZEXT, 在计算 mulh 时要手动进行 SEXT
- a function call must have return address in rd
- a ret shouldnt have offset
- a jal cant be a ret
- strncat 必然末尾补 1 个 0 和 strncpy 只是不够补足 0
- 变长参数会用了
- make 时指定 NEMUFLAGS 给 nemu 传参
- 原来队列这么实现

问题 未解明

- 手册的 2.0 前言说 jal 改成了 U 类型, 但是编译器生成的还是 J 类型
- difftest 中 reg 真的有要求寄存器以特定顺序排列吗?

- TIMER 真的有坑吗? 我看了三遍也没发现而且运行均正常

问题 已解明

- elf 文件的组织格式就是 elf.h 中的结构体的格式, 嗯读发现没出逝确定的
 - sdl 和 nemu 的读写是异步多线程的, 读到的 count 不一定对, 在 am 和 nemu 里都加上队列容量等待以缓解 buffer overflow
-

思考题

- 立即数编码: 先加载到高位在加载低位 or 先加载到寄存器然后寄存器和立即数相加
 - 伪指令是指令的特殊变量限定形式
 - AM 更类似于驱动 (?)
 - 符号是固定且常用的内存空间的表示, 方便寻址. 宏就是文本替换, 局部变量不会大范围访问且不固定没必要成符号
 - 尾函数调用优化
 - 符号是给编译器看的, 运行时直接看地址和偏移
 - 提供 klib 中对应函数的定义, 使编译器不会默认去找 stdlib
 - 如果当前的状态在以前出现过就是死循环
 - 从 reg 中读到旧的值而非内存中最新的值
 - #define _GNU_SOURCE
 - 读到 down 但没有读到 up 说明 key 还在按着
-